NLDesign: A UI Design Tool for Natural Language Interfaces

Tianhao Zhang
zhangtianhao23@mails.ucas.ac.cn
Institute of Software, Chinese Academy of Sciences, University of Chinese Academy of Sciences
Haidian Qu, Beijing Shi, China

Peiguo Fu
fupeiguo@gmail.com
Institute of Software, Chinese Academy of Sciences
Haidian Qu, Beijing Shi, China

Jie Liu*
liujie2016@iscas.ac.cn
Institute of Software, Chinese Academy of Sciences
Haidian Qu, Beijing Shi, China
Guangxi Key Laboratory of Automatic Detecting Technology and Instruments (Guilin University of Electronic Technology)
Gui Lin, Guangxi, China

Yihe Zhang
zhangyihetony@163.com
Institute of Software, Chinese Academy of Sciences, University of Chinese Academy of Sciences
Haidian Qu, Beijing Shi, China

Xingmei Chen
chenxingmei@beyondsoft.com
Beyondsoft
Haidian Qu, Beijing Shi, China

ABSTRACT

Customizing software interfaces often requires designers to possess programming and development skills, resulting in a high technical barrier. We propose a method for UI design based on multi-turn natural language dialogue using generative AI technology. We develop a UI design tool that supports natural language, allowing users to control the layout and properties of UI elements through natural language commands. Users can also define the interaction logic of UI components, enabling personalized customization of software interfaces through natural language descriptions. Experiments demonstrate that this tool enables rapid UI construction using natural language, significantly reducing the user threshold for UI customization and enhancing software adaptability and user experience.

*Corresponding author.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ACM-TURC '24, July 05–07, 2024, Changsha, China
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1011-7/24/07
https://doi.org/10.1145/3674399.3674455

CCS CONCEPTS

• **Human-centered computing → Human computer interaction (HCI)**.

KEYWORDS

UI Customization, Natural Language Interface Design, Interaction Behaviour Definition

ACM Reference Format:
Tianhao Zhang, Peiguo Fu, Jie Liu, Yihe Zhang, and Xingmei Chen. 2024. NLDesign: A UI Design Tool for Natural Language Interfaces. In *ACM Turing Award Celebration Conference 2024 (ACM-TURC '24), July 05–07, 2024, Changsha, China.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3674399.3674455

1 INTRODUCTION

The user interface (UI) is the interface through which applications interact with users, influencing the appearance, functionality, and interaction behavior of software, thereby crucially impacting user experience. Traditional UI design processes often require professionals with UI design tools and programming skills, typically in a one-time process[10, 11]. This can lead to issues such as a lack of personalized customization in software interfaces, high user learning costs, and difficulty in meeting diverse user needs. Users themselves often have the best understanding of their requirements, making user-defined and implemented software interfaces a key solution to these problems. With the rapid development of generative large models in natural language

understanding and code generation[13, 15], we propose a natural language-based UI design method, constructing a tool that allows users to control UI elements' layout and properties through natural language descriptions.

2 RELATED WORK

As deep learning technology advances, natural language processing (NLP) has gradually been applied to UI design[1–3, 9], resulting in UI design products supporting natural language input(see Figure 1). Traditional UI design tools like Figma[4] support user interaction behaviour definition and component generation but do not support natural language interaction. TeleportHQ[5] enables users to generate interfaces using natural language and export them in various formats. Uizard[6] supports generating interfaces and components using natural language and provides component reuse capabilities. Visily[7] supports not only generating interfaces but also editing page elements using natural language. Vercel's v0[8] allows users to generate interface prototypes and edit page elements using natural language, but lacks support for interaction behaviour definition and component generation. The comparison of these tools highlights the potential of deep learning in UI design, while also indicating current limitations in user experience and functionality diversity.

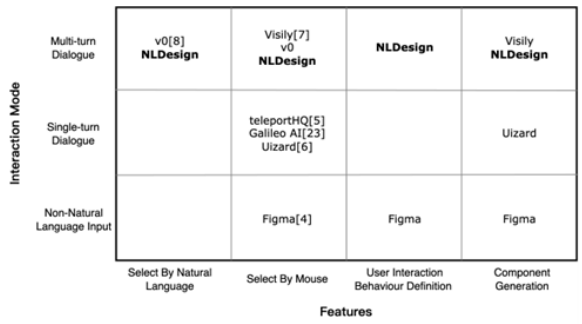


Figure 1: Comparison between UI design tools, parameterized by interaction mode and features.

3 TECHNICAL ARCHITECTURE

We propose a dialogue-based architecture(see Figure 2) that categorizes natural language interface design tasks into component generation and code generation. Users can select elements on a webpage through mouse clicks and input natural language commands to generate or modify UI. In component generation tasks, the system inserts the HTML of the component into the corresponding position in the UI's HTML and re-executes code affecting the element's state but not reflected in the Document Object Model(DOM) structure. Code generation tasks are divided into preprocessing, code generation, and code execution. In the preprocessing stage, the system performs preprocessing to obtain HTML representing the selected interface. In the code generation stage, the system uses Langchain module to embed user commands and selected UI HTML into a predefined template. Combining previous dialogue history, a pre-trained dialogue model generates target code as output. Finally, the system executes the generated code to accomplish user-input tasks.

4 KEY IMPLEMENTATION TECHNOLOGIES

4.1 Natural Language Parsing Module

We build a natural language parsing module that allows users to convert natural language into JavaScript code. This module is based on the Langchain framework[16–18, 22], a platform for developing language model-based applications. It takes user-input natural language commands and HTML strings of selected elements, generating corresponding JS code and task type information as output.

The module consists of three components: prompt templates, chat model, and JSON parser. Prompt templates are pre-defined command formats that transform raw user inputs into better LLM inputs. The module's input is embedded into prompt templates to obtain prompts. Prompts and historical information are input into the chat model to achieve multi-turn dialogue. The chat model, in our system, is gpt-3.5-turbo. To limit the model output format, we bind a specific function schema to the chat model. The schema conveniently specifies which keys the model output JSON object should contain and the corresponding content of each key. To parse the JSON object's output, we use the built-in JSON parser, extracting key-value pairs to facilitate formatted information retrieval from the large model output.

4.2 Element Component Generation Module

This module accomplishes a series of tasks related to generating and editing page elements. It enables users to generate HTML elements using natural language, making web design more intuitive and convenient without requiring knowledge of HTML syntax and details. For example, a user can input "add a title" and the system will automatically generate an h1 tag and insert it into the webpage.

Firstly, user clicks on a specific location on the webpage, and the system retrieves the corresponding HTML element for that location. System assigns a unique index to each element in the entire DOM tree, enabling the generated code to uniquely locate and operate on any element in the DOM tree through indexing.

Next, the system duplicates a copy of the DOM tree and simplifies it by removing unnecessary attributes such as

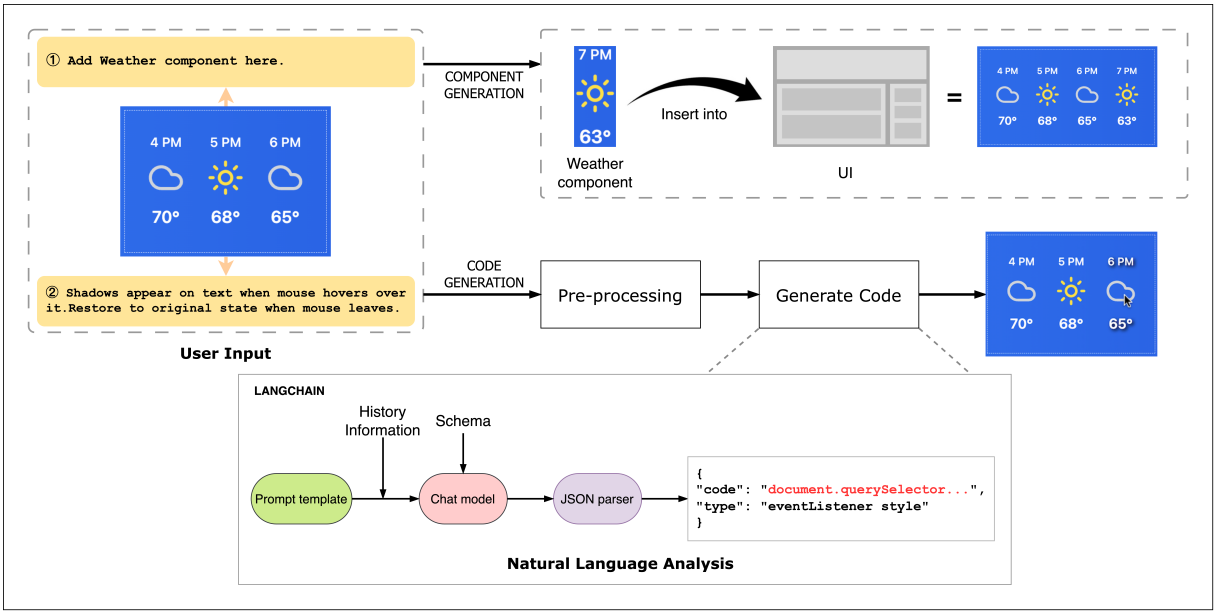


Figure 2: a) User selects page element and requires adding a weather component in it. Our system retrieves the corresponding pre-saved component and inserts it into the UI to generate a new UI. b) User selects an element and adds interaction behaviour. Our system preprocesses the selected HTML and feeds both the user input and HTML into the natural language analysis module, which subsequently generates code. The system then executes the generated code to accomplish the specified task.

class. This aims to reduce the length of the HTML string, avoiding exceeding the model's token input length limit and simplifying the natural language module's analysis.

Finally, the system takes the user's natural language command and the simplified HTML string as input, feeding them into the natural language module for analysis to obtain the target code and other relevant information. The system executes the target code to complete the element generation task, providing users with a new way to create or modify webpage content.

4.3 Natural Language-defined Interaction Behaviour

In addition to element generation, the system also supports users in adding interaction behaviour to interface elements using natural language[12]. This allows users to intuitively and conveniently customize the interaction behavior of elements without needing to understand the technical implementation.

The implementation method for interface interaction is similar to element generation(see Figure 3). The system takes preprocessed user-selected HTML and command input in natural language to the natural language parsing module. This module generates JavaScript code based on HTML and commands. And the system, by executing the generated code, alters the style and layout or adds interaction behaviour to page elements.

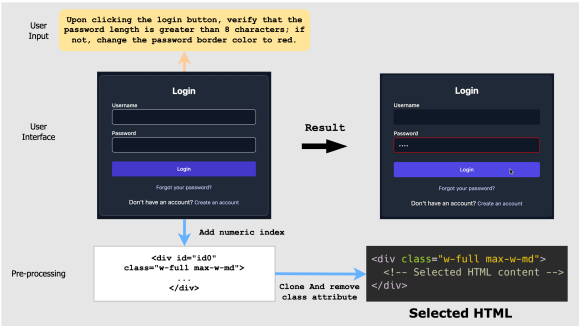


Figure 3: This example illustrates the process of incorporating password verification functionality for the login button. The system preprocesses the selected HTML by adding numeric indexes and removing redundant attributes. Subsequently, the system inputs user instructions and the preprocessed HTML into the natural language parsing module, which produces code that fulfills the user's requirements. The system achieves the user's task through the execution of the generated code.

4.4 Element Selection Method Compatible with Natural Language and Pointing

To enhance user interaction, the system offers a method that is compatible with both natural language and pointing for element selection. This method allows users to select page elements in two ways: mouse clicking or natural language. User can flexibly choose and operate webpage content based on their preferences and needs. For instance, a user can use a mouse click to select a paragraph and then use natural language to specify the style to be modified, such as "change the font to red." Alternatively, a user can use natural language to select a button, such as "change the text content of the login button to black," thus performing the corresponding operation.

The advantage of this element selection method is its combination of the strengths of mouse clicking and natural language. Mouse clicking allows precise selection of target elements, avoiding ambiguity and uncertainty associated with natural language. Natural language enables convenient selection of complex elements, avoiding the cumbersome and challenging nature of mouse clicking.

To implement element selection, user first uses the mouse to select several elements, determining the approximate range of target elements. Subsequently, the user inputs a natural language command to execute the command. The system concatenates the selected multiple elements to obtain the selected HTML. Subsequent process is similar to element generation, where the system feeds HTML and natural language commands into the natural language parsing module to obtain the required code. Due to the possibility of natural language commands selecting multiple complex elements, leading to generated code exceeding token length limits, we limit code usage to concise DOM APIs in prompts. Additionally, unique indexes are used to select elements, ensuring generated code is concise and accurate.

4.5 Composite Component Generation Method

To reuse the modified interface using natural language, we designed components to store page elements and their logic[19–21]. This concept is borrowed from modern frontend frameworks, where components encapsulate page, logic, and style. In our system, we use components to store HTML elements and related interaction behaviour. After selecting HTML elements, user can save their states. Later, user selects where to insert elements and input the component name. System then inserts the saved component into the page. Through component generation, user can save a single element as a component and extend it into more complex components by adding other elements. Ultimately, user can build a complete UI by combining these components(see Figure 4).

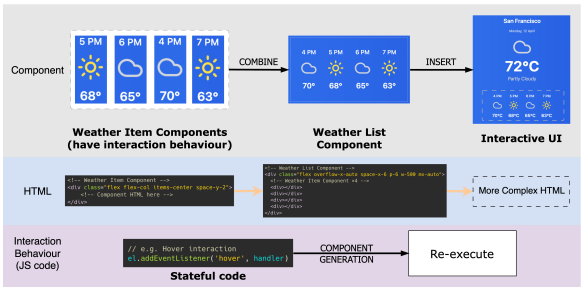


Figure 4: In this example, user can build more sophisticated weather list components by using basic weather item components, ultimately creating the UI. HTML of basic components will be concatenated to obtain the HTML of composite component, thereby constructing a complex UI. After the component is generated, stateful code will be re-executed to add interaction behaviour.

To implement this functionality, we divide user-selected page elements into two parts: HTML strings and associated behaviour. HTML contains the page's structure and inline styles, while the behaviour part includes code generated by user's previous instructions. For example, when the user generates code for an event listener through a command to create a shadow when hovering over text, the code needs to be saved as part of the component to display the correct effect.

In practice, the system categorizes the generated code based on user instructions into three types: the first type has no impact on page element states; the second type affects element states by modifying the DOM structure, such as adding or deleting elements, or changing inline styles; the third type also affects element states, but the modification is not reflected in the DOM structure.

For the save component function, the first type of stateless code does not need to be saved. Although the second type of code affects element states, it does not need to be re-executed since HTML has been saved. However, the third type of code, which affects element states but is not reflected in the DOM structure, needs to be saved. When generating a component, all relevant codes affecting the component are saved as a property, ensuring that they can be executed again.

5 EXPERIMENTS

To assess the effectiveness and efficiency of using natural language for UI design, we designed and conducted a series of experiments. These experiments were divided into two main parts: the first part focused on evaluating the time required for single-turn and multi-turn dialogues in generating

UI elements, while the second part concentrated on assessing the accuracy when using single and multiple selection commands.

5.1 Experimental Setup and Procedure

To ensure that the system effectively assists end-users in UI design, we invited 10 colleagues with no prior front-end development experience to participate in the experiments, randomly assigning them to two experimental groups. All participants performed two experiments on the same page of a weather application. In the first experiment, both groups used single-turn and multi-turn dialogues, respectively, to complete specified UI design tasks, and the time required to complete the tasks was recorded. In the second experiment, both groups were asked to use natural language to perform single or multiple selection tasks on page elements, and the accuracy of the system in executing tasks was statistically analyzed.

The content of Experiment 1 included: 1) Adding a text segment with the content "hover me"; when the mouse hovers over the text, it turns red. 2) Adding a button with the text "click me"; when the button is clicked, a greeting prompt pops up. 3) Adding a table with seven items, representing Sunday to Saturday; adding a button with the content "random"; when the button is clicked, the table items are randomly rearranged. 4) Adding a text "red" with the initial color being red; adding a button "Change color"; when the button is clicked, the text color changes to green, the content changes to "green," and clicking again restores the original state.

Experiment 2 for single selection included: Choosing today's weather, selecting today's date, choosing today's temperature and selecting the city of residence. For multiple selections, the tasks included: Choosing the temperature from 5 pm to 7 pm, selecting all temperature texts and choosing the temperature from Tuesday to Friday.

5.2 Experimental Results

The results of the first experiment, as shown in Figure 5, indicate that single-turn dialogues are more efficient than multi-turn dialogues in generating UI elements. Users can express all information in a single sentence, obtaining results at once and reducing the waiting time between interactions.

In the second experiment, the accuracy for single-selection tasks was 90%, while the accuracy for multiple-selection tasks was 76.67%. For single-selection tasks, occasional errors occurred due to the randomness in the model's prediction process, despite similar user inputs. In multiple-selection tasks, there were instances of selecting the wrong option, such as mistakenly choosing temperature as time in the "Choose temperature from 5 pm to 7 pm" task. This indicates

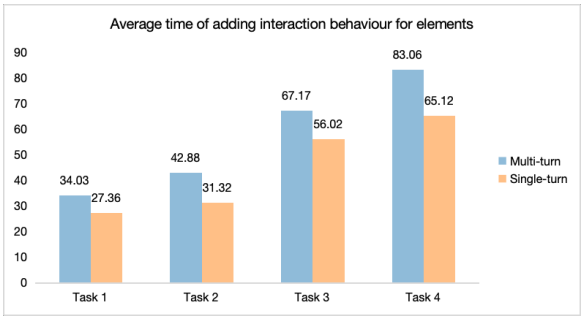


Figure 5: Duration of engagement with the task of adding interaction behavior in both single-turn and multi-turn dialogues.

a limitation in the model's ability to categorize text solely based on HTML strings.

6 DISCUSSION

In contemporary UI design, existing tools support interface generation through natural language, but limitations persist in editing elements and incorporating interaction behavior using natural language. This constraint impedes non-developers from fully expressing creativity in personalized UI customization. We introduce a natural language-based UI design tool enabling users to generate, modify, and add interaction behavior to elements. Users can efficiently save and reuse elements as components. Experimental validation demonstrates the system's usability, highlighting the significant reduction in the UI design threshold and enhancing software adaptability and user experience.

ACKNOWLEDGMENTS

This work was supported by STI2030-Major Projects-2022ZD0211800, Guangxi Key Laboratory of Automatic Detecting Technology and Instruments (No.YQ24206).

REFERENCES

[1] Arkhangelskaya, E.O., Nikolenko, S.I. Deep Learning for Natural Language Processing: A Survey. J Math Sci 273, 533–582 (2023). https://doi.org/10.1007/s10958-023-06519-6

[2] Kshirsagar, Dr.Pravin & Reddy, Dhoma & Dhingra, Mallika & Dhabliya, Dharmesh & Gupta, Ankur. (2022). A Review on Application of Deep Learning in Natural Language Processing. 1834-1840. 10.1109/IC3I56241.2022.10073309.

[3] Siddharth, L & Blessing, Lucienne & Luo, Jianxi. (2021). Natural Language Processing in-and-for Design Research.

[4] Figma. Figma: The Collaborative Interface Design Tool, 2016, https://www.figma.com/

[5] TeleportHQ. TeleportHQ: Low-code Front-end Design & Development, 2017, https://teleporthq.io/

[6] Uizard. Uizard: App, Web and Ul Design Made Easy, 2018, https://uizard.io/

[7] Visily. Visily: Al-Powered Wireframing and Design, 2022, https://www.visily.ai/
[8] v0. v0: Generative UI, 2023, https://v0.dev/
[9] Tae Soo Kim, DaEun Choi, Yoonseo Choi, and Juho Kim. 2022. Stylette: Styling the Web with Natural Language. In Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22). Association for Computing Machinery, New York, NY, USA, Article 5, 1–17. https://doi.org/10.1145/3491102.3501931
[10] H. Dave, S. Sonje, J. Pardeshi, S. Chaudhari and P. Raundale, "A survey on Artificial Intelligence based techniques to convert User Interface design mock-ups to code," 2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS), Coimbatore, India, 2021, pp. 28-33, doi: 10.1109/ICAIS50930.2021.9395994.
[11] B. B. Adefris, A. B. Habtie and Y. G. Taye, "Automatic Code Generation From Low Fidelity Graphical User Interface Sketches Using Deep Learning," 2022 International Conference on Information and Communication Technology for Development for Africa (ICT4DA), Bahir Dar, Ethiopia, 2022, pp. 1-6, doi: 10.1109/ICT4DA56482.2022.9971204.
[12] Zhang, G., Bortoletto, M., Hu, Z., Shi, L., Bâce, M., Bulling, A. (2023). Exploring Natural Language Processing Methods for Interactive Behaviour Modelling. In: Abdelnour Nocera, J., Kristín Lárusdóttir, M., Petrie, H., Piccinno, A., Winckler, M. (eds) Human-Computer Interaction – INTERACT 2023. INTERACT 2023. Lecture Notes in Computer Science, vol 14144. Springer, Cham. https://doi.org/10.1007/978-3-031-42286-7_1
[13] Kristian Kolthoff. 2020. Automatic generation of graphical user interface prototypes from unrestricted natural language requirements. In Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE '19). IEEE Press, 1234–1237. https://doi.org/10.1109/ASE.2019.00148
[14] Pavel Andreevich Samsonov, Johannes Schöning, and Brent Hecht. 2015. A User Interface for Encoding Space Usage Rules Expressed in Natural Language. In Proceedings of the 33rd Annual ACM Conference Extended Abstracts on Human Factors in Computing Systems (CHI EA '15). Association for Computing Machinery, New York, NY, USA, 2211–2216. https://doi.org/10.1145/2702613.2732737
[15] Yu Su, Ahmed Hassan Awadallah, Miaosen Wang, and Ryen W. White. 2018. Natural Language Interfaces with Fine-Grained User Interaction: A Case Study on Web APIs. In The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval (SIGIR '18). Association for Computing Machinery, New York, NY, USA, 855–864. https://doi.org/10.1145/3209978.3210013
[16] Yu Cheng, Jieshan Chen, Qing Huang, Zhenchang Xing, Xiwei Xu, and Qinghua Lu. 2023. Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains. ACM Trans. Softw. Eng. Methodol. Just Accepted (December 2023). https://doi.org/10.1145/3638247
[17] Zhen Zeng, William Watson, Nicole Cho, Saba Rahimi, Shayleen Reynolds, Tucker Balch, and Manuela Veloso. 2023. FlowMind: Automatic Workflow Generation with LLMs. In Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF '23). Association for Computing Machinery, New York, NY, USA, 73–81. https://doi.org/10.1145/3604237.3626908
[18] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. Proc. ACM Program. Lang. 7, PLDI, Article 186 (June 2023), 24 pages. https://doi.org/10.1145/3591300
[19] Christian Brel, Philippe Renevier Gonin, Alain Giboin, Michel Riveill, and Anne-Marie Dery. 2014. Reusing and Combining UI, Task and Software Component Models to Compose New Applications. In Proceedings of the 28th International BCS Human Computer Interaction Conference on HCI 2014 - Sand, Sea and Sky - Holiday HCI (BCS-HCI '14). BCS, Swindon, GBR, 1–10. https://doi.org/10.14236/ewic/hci2014.1
[20] Nishit Gajjar, Vinoth Pandian Sermuga Pandian, Sarah Suleri, and Matthias Jarke. 2021. Akin: Generating UI Wireframes From UI Design Patterns Using Deep Learning. In 26th International Conference on Intelligent User Interfaces - Companion (IUI '21 Companion). Association for Computing Machinery, New York, NY, USA, 40–42. https://doi.org/10.1145/3397482.3450727
[21] Heikki Keränen and Johan Plomp. 2002. Adaptive runtime layout of hierarchical UI components. In Proceedings of the second Nordic conference on Human-computer interaction (NordiCHI '02). Association for Computing Machinery, New York, NY, USA, 251–254. https://doi.org/10.1145/572020.572058
[22] Savvas Petridis, Michael Terry, and Carrie Jun Cai. 2023. PromptInfuser: Bringing User Interface Mock-ups to Life with Large Language Models. In Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23). Association for Computing Machinery, New York, NY, USA, Article 237, 1–6. https://doi.org/10.1145/3544549.3585628
[23] Galileo Al. Galileo Al: Copilot for interface design, 2021, https://www.usegalileo.ai/